Classical Circuit Calculator

Iteration 02: System Design and Implementation Start
Eric Sebastian and Joshua Marquez
Progress update: October 30, 2026

1. Scope and leadership

Our project is a C++ calculator for basic DC resistive circuits. The user chooses a calculation, enters known values, and gets an answer with units. We included resistance, current, power, series resistance, and parallel resistance. We are not covering AC circuits or complicated networks in this version.

Joshua Marquez is the project leader for this iteration because Eric led the previous one. His role is to coordinate progress, maintain the task table, keep responsibilities balanced, and present the team status at the check-in.

2. Implementation progress

All five calculation options and the repeating menu are implemented. Eric set up and tested the starter in WSL/VS Code, committed the project files, and pushed the iteration-2-starter branch. Joshua submitted his implementation pull request for power, parallel resistance, and the menu. Eric documented his review. Joshua resolved merge conflicts, and the team confirmed that the pull request was merged into main.

Final verification: the starter passed 16 tests. After integration, the latest recorded suite reported 22 passed and 11 failed. Test-file handling has been corrected, and expected output labels are being aligned without changing expected numbers. A passing rerun of the merged version is still pending.

3. Menu and input rules

Option	Calculation	Input rule
1	Resistance: R = V / I	V >= 0 and I > 0
2	Current: I = V / R	V >= 0 and R > 0
3	Power: P = V x I	V and I >= 0
4	Series: Req = sum of R	1-100 positive resistors
5	Parallel: Req = 1 / sum(1/R)	1-100 positive resistors
0	Exit	End the program

Use volts, amperes, and ohms. Enter one finite number at each prompt. Invalid input returns to the menu. Answers use four decimal places. This version uses nonnegative magnitudes and does not model ideal zero-ohm resistors.

4. Task table

Done below means implementation is present. The shared final test check is tracked separately under T7. Assigned responsibility follows the original plan.

ID / menu	Task / owner	Files and functions	Deliverable / status
T1 / 1	Resistance / Eric	src/tasks.cpp: resistanceTask src/calculator.cpp: resistance	R = V / I, ohms done
T2 / 2	Current / Eric	src/tasks.cpp: currentTask src/calculator.cpp: current	I = V / R, A done
T3 / 3	Power / Joshua	src/tasks.cpp: powerTask src/calculator.cpp: power	P = VI, W done
T4 / 4	Series / Eric	src/tasks.cpp: seriesTask src/calculator.cpp: series	Sum of resistor values done
T5 / 5	Parallel / Joshua	src/tasks.cpp: parallelTask src/calculator.cpp: parallel	Reciprocal sum done
T6 / 0-5	Menu / Joshua	src/main.cpp: main src/tasks.cpp: readMenuChoice	Switch menu and exit done
T7 / N/A	Testing / Both	test.sh and tests/	Align labels and verify final run in progress
T8 / N/A	Documentation / Both	README.md and docs/	Updated report and Excel; upload pending in progress
T9 / N/A	Review / submission / Both	GitHub and Canvas	Review recorded and merge completed; submission pending in progress

Declarations are in src/tasks.h and src/calculator.h. Eric remains responsible for resistance, current, and series calculations. Joshua remains responsible for power, parallel resistance, and the menu. Both share testing, documentation, and review.

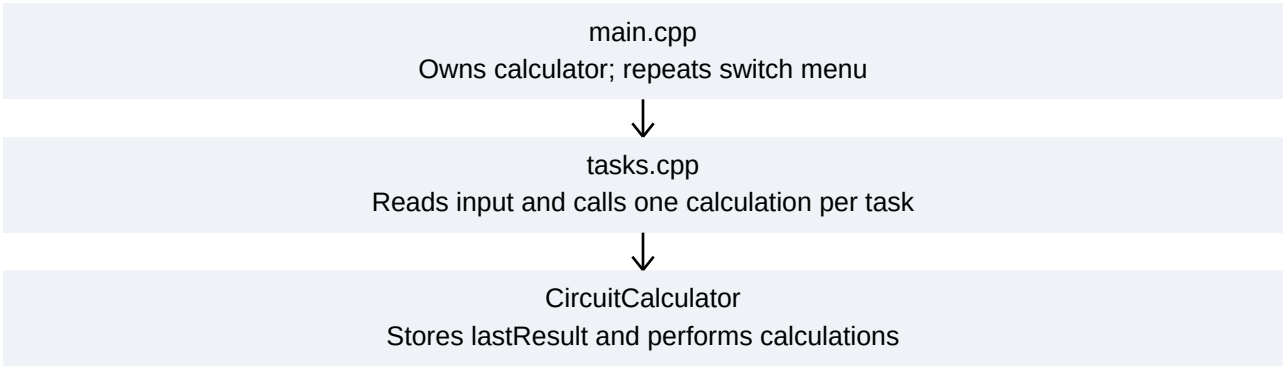
5. Testing and integration

The saved tests cover the five formulas, decimal rounding, zero power, zero denominators, invalid resistor values, nonnumeric input, menu choices, resistor counts, extra text, overflow, and repeated calculations. Joshua also added parallel cases with three and unequal resistors.

Integration kept two sets of test files with different expected-file extensions. The runner was updated to recognize .expect and .expected files and fail for missing or empty expected files. The output labels also differed, such as Current: versus Current =. The visible parallel cases produced the expected numbers, 2.0000 and 0.5714 ohms. Final suite success will be recorded after the rerun.

6. System design

CircuitCalculator contains private **double lastResult**, initialized to zero. Its private **save(double)** method validates and stores a finite result. The public calculation methods are **resistance(voltage, current)**, **current(voltage, resistance)**, **power(voltage, current)**, **series(resistors)**, and **parallel(resistors)**. The design also provides **getLastResult() const**. Resistor lists use vector<double>. Calculation methods return double. Invalid values are reported with exceptions.



File	Purpose
src/main.cpp	Main loop, switch menu, exception handling
src/tasks.h and src/tasks.cpp	Task declarations, input prompts, validation
src/calculator.h and src/calculator.cpp	Calculator class, private state, formulas
tests/ and test.sh	Saved inputs, expected results, test runner
README.md	Build, run, scope, and usage instructions
docs/	PDF report and Excel project plan

7. Build and run

Open a terminal in the repository folder. Create the build folder, compile the source files, and run the program:

```
mkdir -p build
g++ -std=c++17 -Wall -Wextra -pedantic src/*.cpp -o build/calculator
./build/calculator
bash test.sh
```

The switch statement calls one task function for each menu option. Each task reads values, calls a calculator method, and displays the result with its unit. The program repeats until exit is chosen or input ends.

8. Contributions

Member	Completed work recorded	Remaining work
Eric Sebastian	Set up the project in WSL/VS Code. Ran the original 16 tests successfully. Manually checked 12 V / 2 A = 6 ohms and exit. Committed and pushed the starter. Reviewed Joshua's pull request. Updated the test runner and prepared label corrections.	Verify the final test run, upload updated documents, and submit the repository link.
Joshua Marquez	Created the shared repository. Submitted the implementation pull request for power, parallel resistance, menu, and exit. Provided a power demonstration. Resolved merge conflicts. The team confirmed the merge.	Confirm review of Eric's pull request, review final test fixes, keep the task plan current, and submit the repository link.

The initial code was prepared with assistance. The entries above record demonstrated setup, testing, GitHub, review, and integration work rather than claiming sole authorship of the starter. Eric's approval was shown on Joshua's pull request. Joshua's approval of Eric's pull request has not been confirmed in the supplied evidence.

9. Revised timeline

Stage	Progress / target
Design and implementation	Completed; all five menu features implemented
Review and integration	Eric review recorded; conflicts resolved; merge confirmed
Final test verification	In progress; align labels and rerun before final verification
Documentation	PDF and Excel updated; revised GitHub upload pending
Submission	Each student submits the repository link separately on Canvas

Work is recorded as of October 30, 2026. The assignment screenshot shows an October 30, 11:59 PM submission cutoff. Earlier milestone dates were not supplied. The sequence above tracks progress without inventing earlier dates.

10. Submission checklist

Finish the test rerun and review the changes. Upload the corrected runner, test expectations, and these revised documents to GitHub. Confirm that main contains src/, tests/, test.sh, README.md, and docs/ with the report and Excel plan. Each student submits the repository URL separately on Canvas. Canvas submission remains pending.

Sources: Iteration 1 project plan, Iteration 2 assignment screenshot, terminal results, pull request screenshots, and team progress supplied in this conversation.